

Figure 3: Comparison of OT Approximation Methods. **Left:** Average OT distance across batch sizes. Minibatch OT (blue) fails to reduce distances much compared to independent coupling (red dash). Equivariant OT (orange) significantly reduces distance values. Our OT approximation is on par with Equivariant OT. **Right:** Computational time for OT across batch sizes. Minibatch OT (blue) maintains a reasonable computational time (~ 1 second) with batch size $B = 256$. Equivariant OT (orange) grows quadratically starting from 2.2 seconds with $B = 1$.

Optimal Transport (OT) Map. In the CFM objective in Eq. 2, the training pair $(\mathbf{x}_0, \mathbf{x}_1)$ is sampled from an independent coupling: $q(\mathbf{x}_0, \mathbf{x}_1) = q_0(\mathbf{x}_0)q_1(\mathbf{x}_1)$. However, Tong et al. (2023); Pooladian et al. (2023) show that we can sample the training pair from any coupling that satisfies the marginal constraint: $\int q(\mathbf{x}_0, \mathbf{x}_1)d\mathbf{x}_1 = q_0(\mathbf{x}_0)$ and $\int q(\mathbf{x}_0, \mathbf{x}_1)d\mathbf{x}_0 = q_1(\mathbf{x}_1)$. They show an optimal transport (OT) map π that minimizes $\int \|\mathbf{x}_0 - \mathbf{x}_1\|^2 \pi(\mathbf{x}_0, \mathbf{x}_1)d\mathbf{x}_0d\mathbf{x}_1$ is a good choice for data coupling, leading to a straighter trajectory. Yet, obtaining the optimal transport map is often difficult for complex distributions. Next, we review two main directions for approximating the OT map:

(i) **Minibatch OT:** Tong et al. (2023); Pooladian et al. (2023) approximate the actual OT by computing it at the batch level. Specifically, they sample a batch of Gaussian noises $\{\mathbf{x}_0^1, \dots, \mathbf{x}_0^B\} \sim q_0$ and data samples $\{\mathbf{x}_1^1, \dots, \mathbf{x}_1^B\} \sim q_1$, where B is the batch size. They solve a discrete optimal transport problem, assigning noises to data samples while minimizing a cost function $C(\mathbf{x}_0, \mathbf{x}_1)$. The cost function is typically the squared-Euclidean distance, *i.e.*, $C(\mathbf{x}_0, \mathbf{x}_1) = \|\mathbf{x}_0 - \mathbf{x}_1\|^2$, and the assignment problem is often solved using the Hungarian algorithm (Kuhn, 1955). After computing the assignment, we can use the assigned pairs to train the vector field network via Eq. 2. As the batch size B approaches infinity, this procedure converges to sampling from the true OT map.

(ii) **Equivariant OT Flow Matching:** Song et al. (2024); Klein et al. (2024) also approximate the OT at the batch level, but they focus on generating elements invariant to certain group G , such as permutations, rotations, and translations. Specifically, they propose replacing the aforementioned cost function $C(\mathbf{x}_0, \mathbf{x}_1)$ with one that accounts for these group elements: $\bar{C}(\mathbf{x}_0, \mathbf{x}_1) = \min_{g \in G} \|\mathbf{x}_0 - \rho(g)\mathbf{x}_1\|^2$, where $\rho(g)$ is the matrix representation of the group element g . This approach significantly reduces the OT distance even with a small batch size, demonstrating success in generating molecular data. Intuitively, using the cost function defined above allows us to align data and noise together (in our case via permutation) when computing the minibatch OT.

So far, we consider generic unconditional generative learning. It is worth noting that mini-batch OT does not easily extend to conditional generation problems, *i.e.*, learning $p(\mathbf{x}|\mathbf{y})$ for a generic input conditioning $\mathbf{y}$, when there is only one training sample $\mathbf{x}$ for each input conditioning $\mathbf{y}$. This is because the OT assumes access to a batch of training samples for each $\mathbf{y}$.

3.2 EXISTING OT APPROXIMATION FOR POINT CLOUD GENERATION

We focus on generating 3D shapes represented as point clouds. A point cloud $\mathbf{x}_1 \in \mathcal{R}^{N \times 3}$ is a set of points sampled from the surface of a shape $\mathcal{S}$, where N is the number of points. Unlike 2D images, point clouds have unique properties that pose challenges for existing OT methods:

(i) **Permutation Invariance.** A point cloud, while arranged in a matrix form, is inherently a set. Shuffling points in $\mathbf{x}_1$ should represent the same shape. Mathematically, given a permutation matrix $\rho(g)$, the sampling probability remains unchanged, *i.e.*, $q_1(\rho(g)\mathbf{x}_1) = q_1(\mathbf{x}_1)$.

(ii) **Dense Point Set.** Point clouds are finite samples on surfaces. However, similar to low-resolution images, sparse point sets may miss fine geometric structures and details. Thus, most works use dense point sets (say $N \geq 2048$) to accurately capture 3D shapes.

Existing approach to estimating OT maps face these challenges on point clouds:

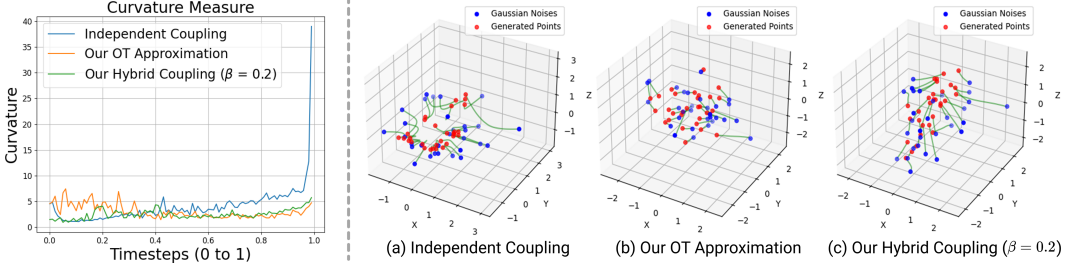


Figure 4: Analysis of trajectory straightness using different couplings to obtain training pairs. Left: We plot the square norm of the difference between successive vector fields, i.e., $\|v_{\theta,t+1}(x_{t+1}) - v_{\theta,t}(x_t)\|$, as a measure of trajectory curvature. Right (a-c): Trajectory samples obtained by models trained with (a) independent coupling, (b) our OT approximation, and (c) our hybrid coupling with $\beta = 0.2$. Note that we subsample the point cloud to 30 points for a better trajectory visualization.

Ineffectiveness of Minibatch OT. Minibatch OT, effective in low-dimensional and image domains, fails for point clouds due to property (i). There are $N!$ equivalent representations of the same point cloud, implying $N!$ equivalent training pairs $(\mathbf{x}_0, \mathbf{x}_1)$. An OT-sampled pair should minimize the cost: $C(\mathbf{x}_0, \mathbf{x}_1) = \min_{g \in G} C(\mathbf{x}_0, \rho(g)\mathbf{x}_1)$. However, in Minibatch OT’s with no permutation, the assignments grow quadratically with batch size, while the number of permutations grows exponentially. As shown in Figure 3 (left), Minibatch OT achieves only about 6% reduction in the cost even with batch size 256, indicating limited effectiveness of this approach in point cloud generation.

Inefficient OT Maps in Equivariant OT. Equivariant OT produces high-quality maps, but is computationally expensive for point cloud generation. Figure 3 (left) shows a 48.7% reduction even with a batch size 1, showing the importance of aligning points and noise via permutation. However, unlike molecular data, which has limited size ($N=55$ in (Klein et al., 2024)), representing 3D shapes needs a larger N , following property (ii). Solving the optimal transport cost takes an $O(B^2 N^3)$ computational complexity because of the quadratic number of noise and point cloud pairs in a batch of B examples, and $O(N^3)$ for the Hungarian algorithm (Kuhn, 1955). Figure 3 (right) shows how this grows rapidly even for a typical point cloud size ($N = 2048$). It takes around 2.2 seconds for the OT computation even for $B = 1$, leading to a significant bottleneck in the training process that is more than 40x slower than independent coupling.

3.3 OUR OT APPROXIMATION

A simple approach to generate training point clouds is to re-sample the points from the object surface in each training iteration. However, most point cloud generation methods avoid this tedious online sampling by pre-sampling a dense point superset $X_1 \in \mathcal{R}^{M \times 3}$ with $M \gg N$. During training, random subsets of X_1 are selected as training targets. This procedure converges to the true sampling distribution, following a straightforward extension of the law of large numbers (see Appendix proposition 1 for details). In a similar spirit, we compute an offline OT map between a dense point superset $X_1 \in \mathcal{R}^{M \times 3}$ and a dense randomly-sampled Gaussian noise superset $X_0 \in \mathcal{R}^{M \times 3}$, and during training, subsample data-noise pairs from the supersets based on the offline OT map.

Superset OT Precomputation. Given supersets X_0 and X_1 , we compute a bijective map Π between them, i.e., $\Pi : X_0 \leftrightarrow X_1$. When M is small, following (Song et al., 2024; Klein et al., 2024), we compute the bijective map using the Hungarian algorithm (Kuhn, 1955). However, this algorithm scales poorly for large point clouds, i.e., $M > 10K$. For such large point clouds, we use Wasserstein gradient flow to transform X_0 into X_1 by minimizing their Wasserstein distance iteratively. Using an efficient GPU implementation (Feydy et al., 2019), the OT precomputation takes only ~ 30 seconds for 100K points, showing its high scalability. See Appendix C for the details in procedure.

Online Random Subsampling. Given precomputed coupling $\Pi(X_0, X_1)$, we randomly sample data-noise pair $(\mathbf{x}_0, \mathbf{x}_1) \sim \Pi(X_0, X_1)$ and we train the flow matching model according to Equation 2. As we show in Figure 3 (left), this significantly reduces the transport cost, while introducing negligible training overhead. In Appendix A.1.2, we show that the sampled training pair converges to correct marginals if M is sufficiently large.

In practice, using these pairs for training results in straighter sampling trajectories, measured by the curvature of the sampling trajectory, as shown in Figure 4 (left). The model trained with our OT approximation (orange curve) exhibits a much lower maximum curvature compared to the one with

independent coupling (blue curve). We also visualize the sample trajectories of these two models in Figures 4 (right) (a-b), confirming straighter trajectories for our model.

3.4 HYBRID COUPLING

Though training flows with OT couplings comes with appealing theoretical justifications (*e.g.*, straight sampling trajectories), we identify a key training challenge with them that is often overlooked in the OT flows literature. Our experiments (*e.g.*, Section 4.1) indicate that flows trained with equivariant OT maps are often outperformed by those with independent coupling in terms of sample quality, especially when the number of sampling steps is large. We hypothesize this is due to the increasing complexity of target vector fields for OT couplings that makes their approximation harder with neural networks.

Intuitively, as we make target sampling trajectories straighter using more accurate OT couplings, the complexity of generation shifts toward smaller time steps. As Figure 2 shows, in the limit of straight trajectories, the learned vector field $\mathbf{v}_{\theta,0}(\mathbf{x}_0)$ should be able to switch between different target point clouds with small variation in $\mathbf{x}_0$, forcing $\mathbf{v}_{\theta,0}$ to be complex at $t=0$. This problem is further exacerbated in the equivariant OT flows with large N where permuting Gaussian noise cloud in the input makes it virtually the same for all target point clouds. To verify this, we measure the trained vector field’s complexity for 3D point cloud generation using the Jacobian Frobenius norm in different timesteps in Figure 5. As hypothesized above, switching from independent coupling (blue curve) to our OT approximation (orange curve) shifts the high Jacobian norm at $t \approx 1$ for independent coupling to $t \approx 0$ for OT coupling. This motivates us to develop a method to make it easier for neural networks to approximate the target vector field, while still maintaining a relatively straight path.

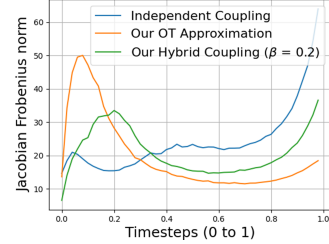


Figure 5: We show Jacobian Frobenius Norm for different trained $\mathbf{v}_{\theta,t}$ over different time intervals, which measures the model complexity as in (Dockhorn et al., 2022).

Hybrid Coupling. Given the different behavior of independent and OT couplings in Figure 5, we aim to reduce the complexity of the vector field at early timesteps by combining our OT approximation with independent coupling. To do so, we propose injecting additional random Gaussian noise into $\mathbf{x}_0$, making our OT couplings even less “optimal”. The new training $\mathbf{x}'_0$ is defined as:

$$\mathbf{x}'_0 = \sqrt{1 - \beta} \mathbf{x}_0 + \sqrt{\beta} \epsilon, \quad \epsilon \sim \mathcal{N}(\epsilon; 0, \mathbf{I}), \quad (3)$$

where $\beta \in [0, 1]$ is the blending coefficient. Intuitively β allows us to switch smoothly between independent and OT couplings. Specifically, for $\beta \rightarrow 0$, the coupled data and noise pairs converge to our OT couplings, whereas when $\beta \rightarrow 1$, they follow the independent coupling.

We empirically observe that $\beta = 0.2$ in most experiments strikes a good balance between learning complexity (as shown by the green curve in Figure 5), low curvature for the sampling trajectories (the green curve in Figure 4 (left) and Figure 4 (right) (c)), and sample generation quality (shown later in Section 4.3). In the next section, we refer to this hybrid coupling as our main method.

4 EXPERIMENT

In this section, we present our experimental results for unconditional and conditional 3D point cloud generation, *i.e.*, shape completion, after reviewing dataset and evaluation details.

Dataset. Following Yang et al. (2019); Klokov et al. (2020); Cai et al. (2020); Zhou et al. (2021), we employ the ShapeNet dataset (Chang et al., 2015) for training and evaluating our approach. Specifically, we train separate generative models for the Chair, Airplane, and Car categories with the provided train-test splits. To form our training point clouds, we randomly sample a superset of $M = 100\text{K}$ points on each input 3D shape. Then, during the online random subsampling, we randomly subsample the superset to $N = 2,048$ points, following the procedure in Section 3.3.

In addition, we prepare a partial input shape for the shape completion task for each training shape. We use the GenRe dataset (Zhang et al., 2018) for depth renderings of ShapeNet shapes. Partial point clouds are obtained by unprojecting and sampling up to 600 points from these depth images.